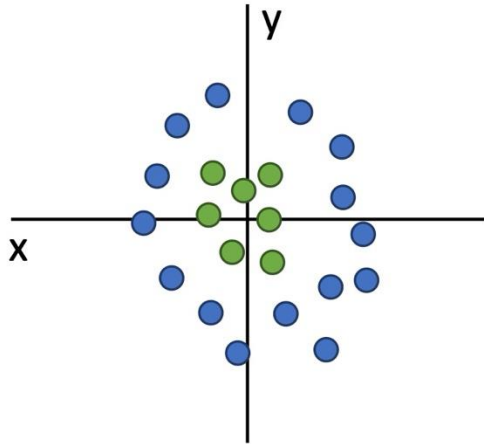


Neural Network

```
lookup.KeyValue  
f.constant(['em  
=tf.constant([G  
lookup.StaticV  
_buckets=5)
```

Limitations of Linear Classifier

Geometric Viewpoint



Visual Viewpoint

One template per class:
Can't recognize different
modes of a class



Feature Transforms

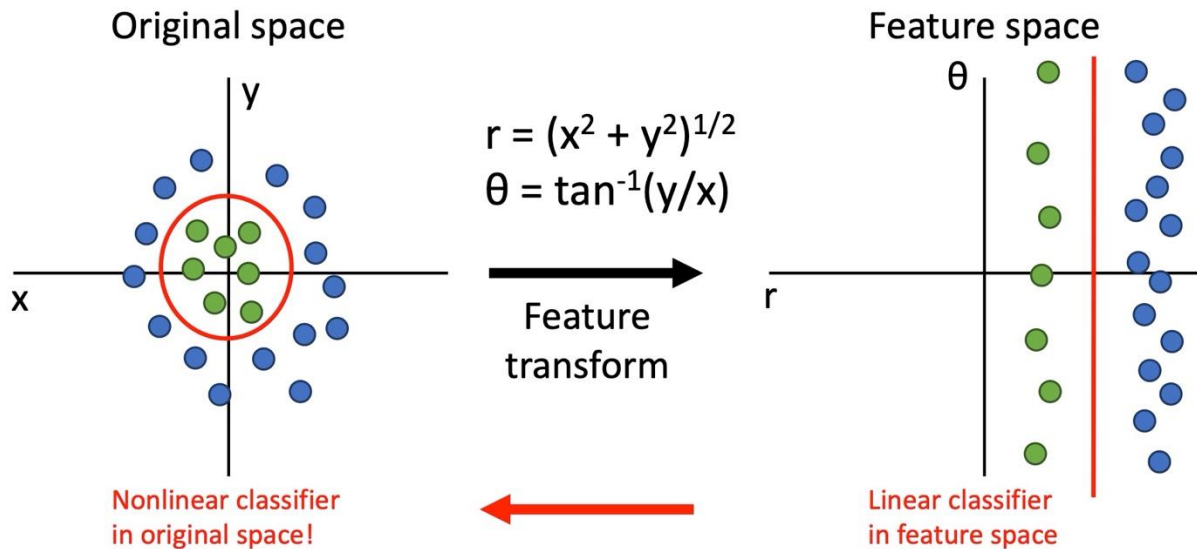


Image Features

Color Histogram

**Histogram of
Oriented Gradients**

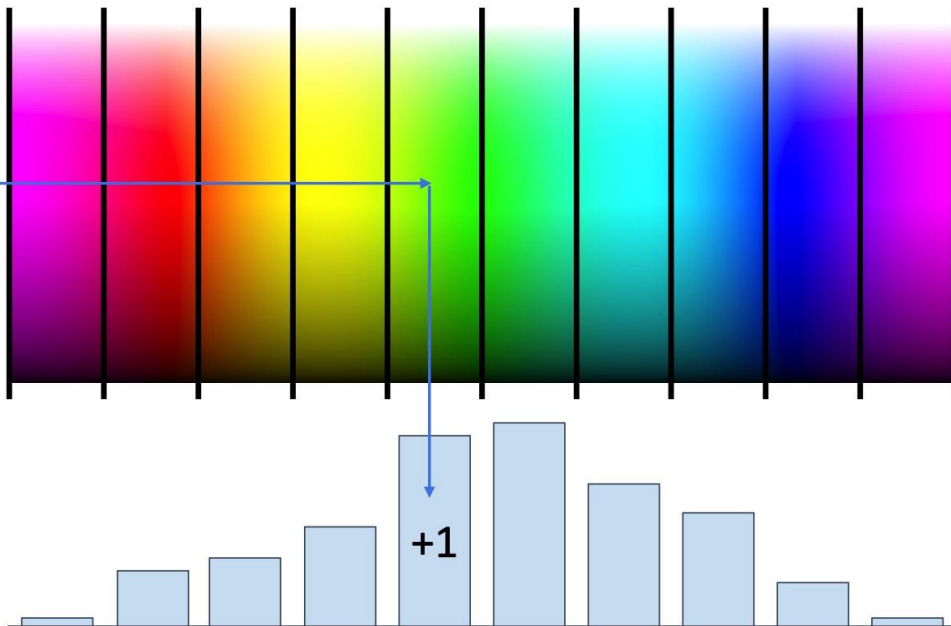
Bag of Words

Image Features

Color



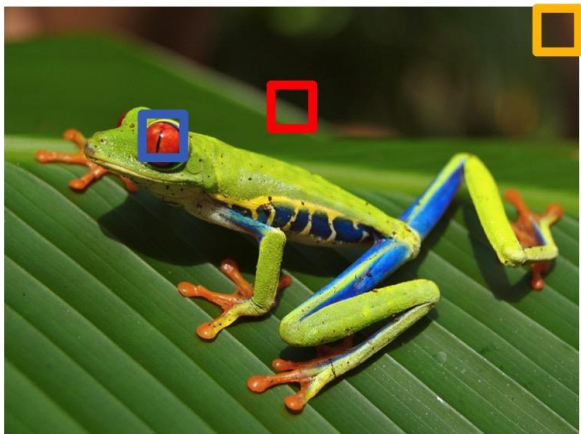
Ignores texture,
spatial positions



Words

Image Features

Col

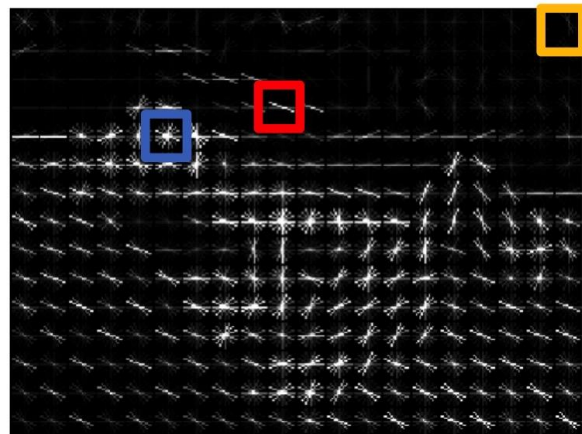


Weak edges

Strong diagonal
edges



Edges in all
directions



ords

1. Compute edge direction / strength at each pixel
2. Divide image into 8x8 regions
3. Within each region compute a histogram of edge directions weighted by edge strength

Example: 320x240 image gets divided into 40x30 bins; 8 directions per bin; feature vector has $30 \times 40 \times 9 = 10,800$ numbers

Lowe, "Object recognition from local scale-invariant features", ICCV 1999
Dalal and Triggs, "Histograms of oriented gradients for human detection," CVPR 2005

Image Features

Step 1: Build codebook



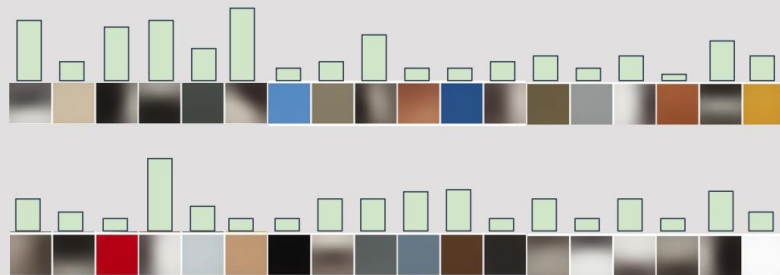
Extract random patches



Cluster patches to form "codebook" of "visual words"



Step 2: Encode images



Fei-Fei and Perona, "A bayesian hierarchical model for learning natural scene categories", CVPR 2005

Image Features

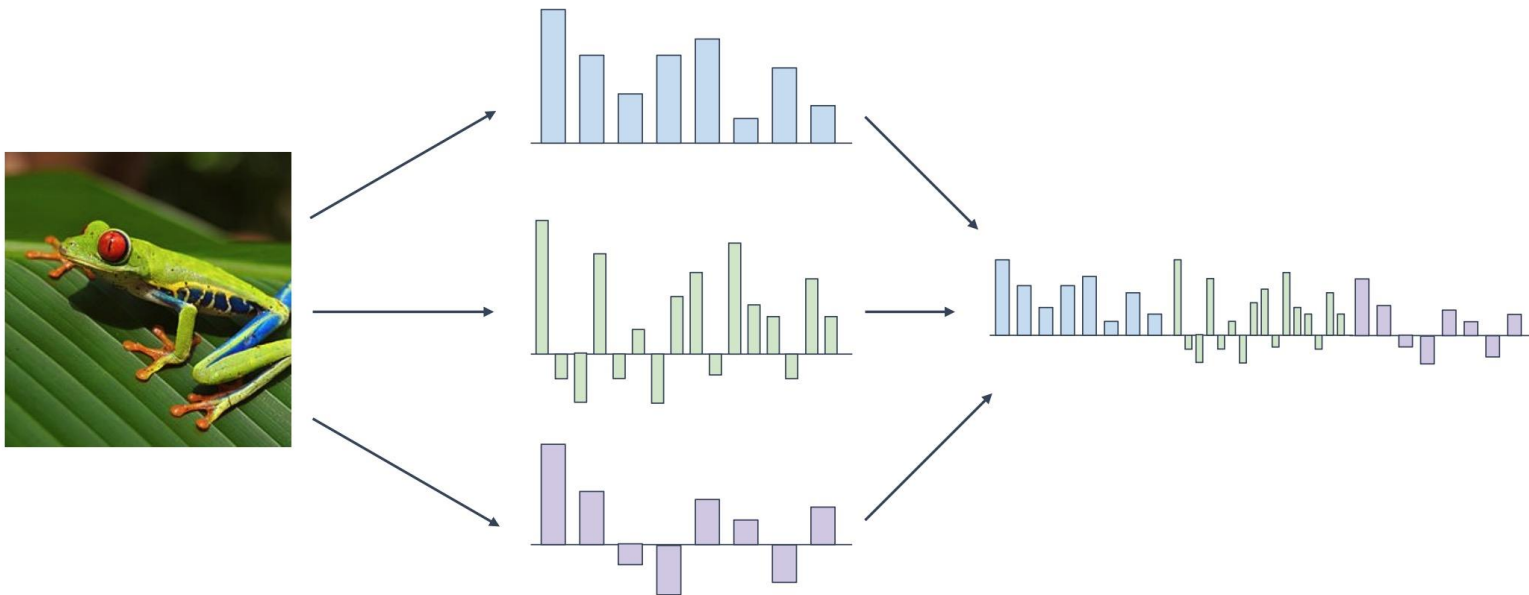
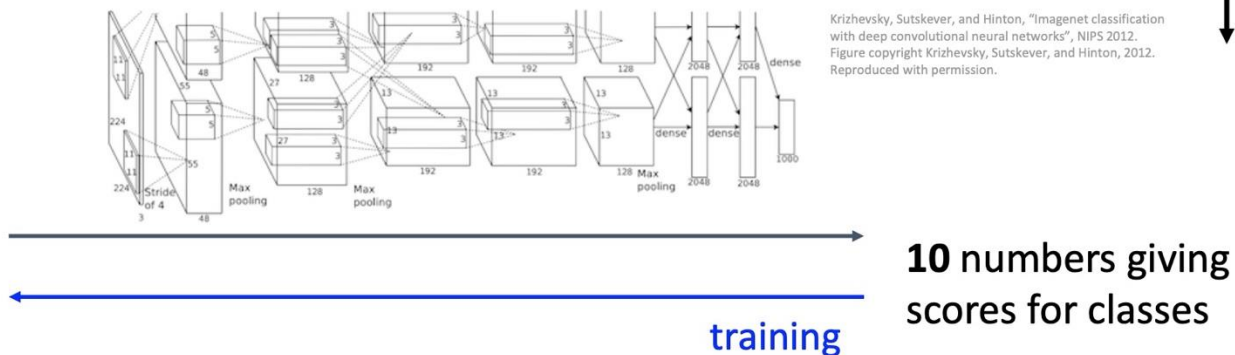
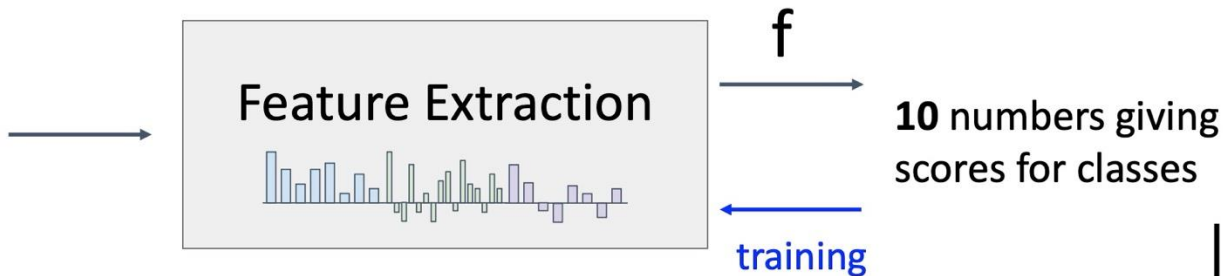


Image Features



Neural Network

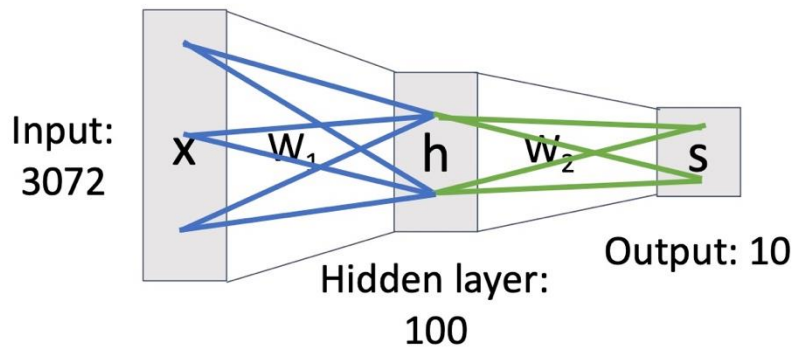
(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$
or 3-layer Neural Network

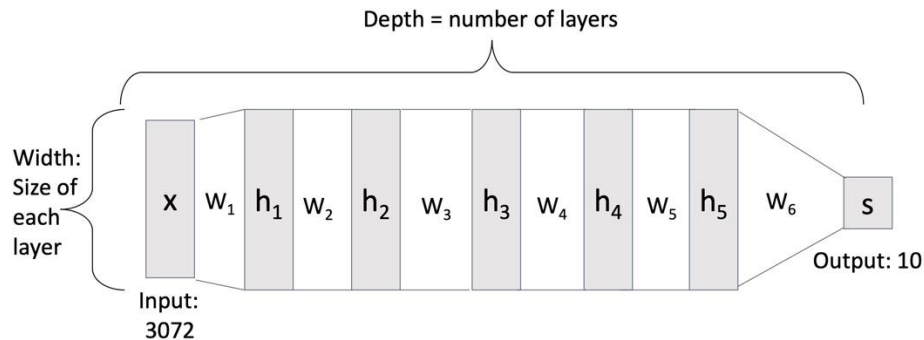
$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

$$W_3 \in \mathbb{R}^{C \times H_2} \quad W_2 \in \mathbb{R}^{H_2 \times H_1} \quad W_1 \in \mathbb{R}^{H_1 \times D} \quad x \in \mathbb{R}^D$$

Neural Network → Deep Neural Networks



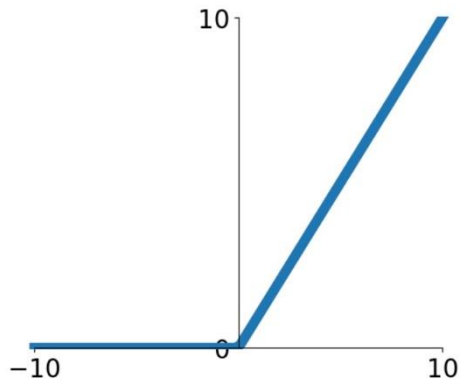
Fully-connected neural network
Also “Multi-Layer Perceptron” (MLP)



$$s = W_6 \max(0, W_6 \max(0, W_5 \max(0, W_4 \max(0, W_3 \max(0, W_2 \max(0, W_1 x))))))$$

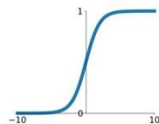
Activation Function

The function $ReLU(z) = \max(0, z)$ is called “Rectified Linear Unit”



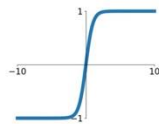
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



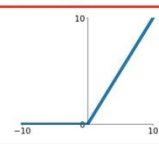
tanh

$$\tanh(x)$$



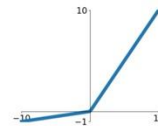
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

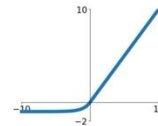


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

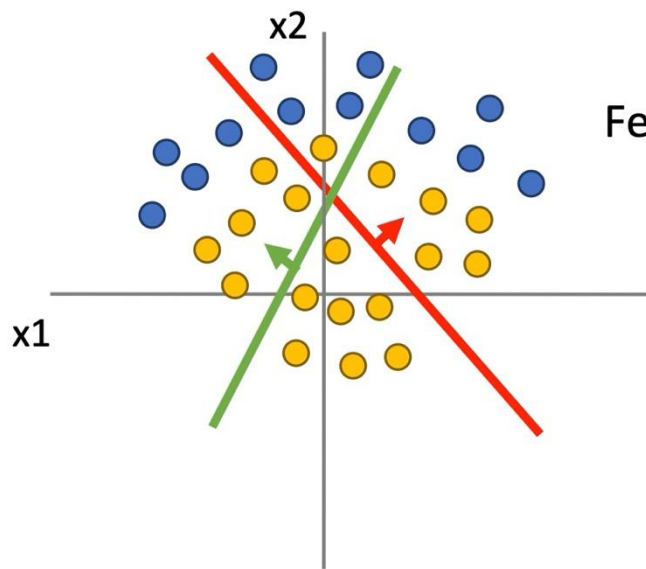


Neural Network

Initialize weights and data	{	4	N, Din, H, Dout = 64, 1000, 100, 10
		5	x, y = randn(N, Din), randn(N, Dout)
		6	w1, w2 = randn(Din, H), randn(H, Dout)
		7	for t in range(10000):
Compute loss (sigmoid activation, L2 loss)	{	8	h = 1.0 / (1.0 + np.exp(-x.dot(w1)))
		9	y_pred = h.dot(w2)
		10	loss = np.square(y_pred - y).sum()
Compute gradients	{	11	dy_pred = 2.0 * (y_pred - y)
		12	dw2 = h.T.dot(dy_pred)
		13	dh = dy_pred.dot(w2.T)
		14	dw1 = x.T.dot(dh * h * (1 - h))
SGD step	{	15	w1 -= 1e-4 * dw1
		16	w2 -= 1e-4 * dw2

Space Warping

Points not linearly separable in original space

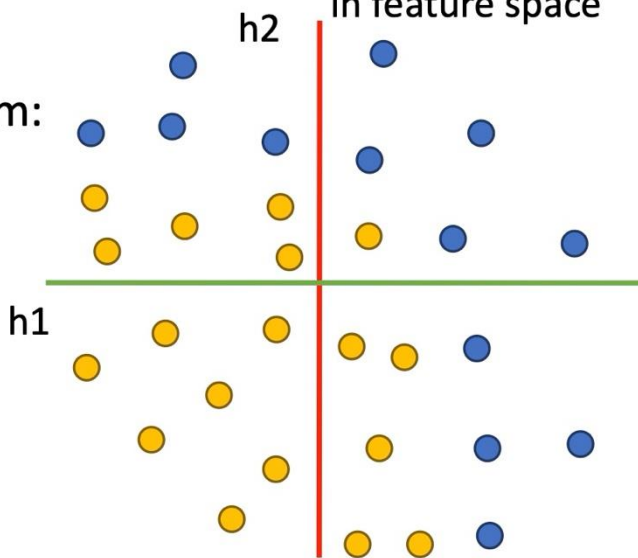


Where x , h are both 2-dimensional

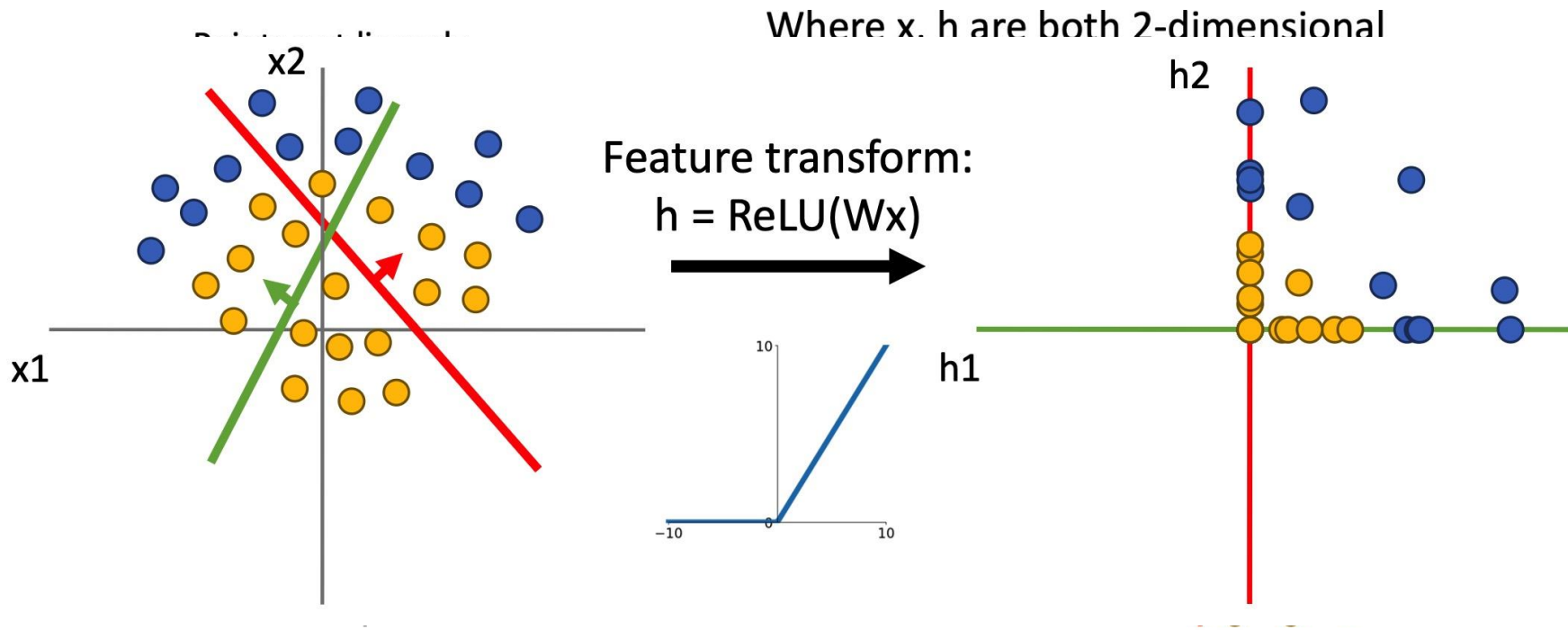
Feature transform:
 $h = Wx$



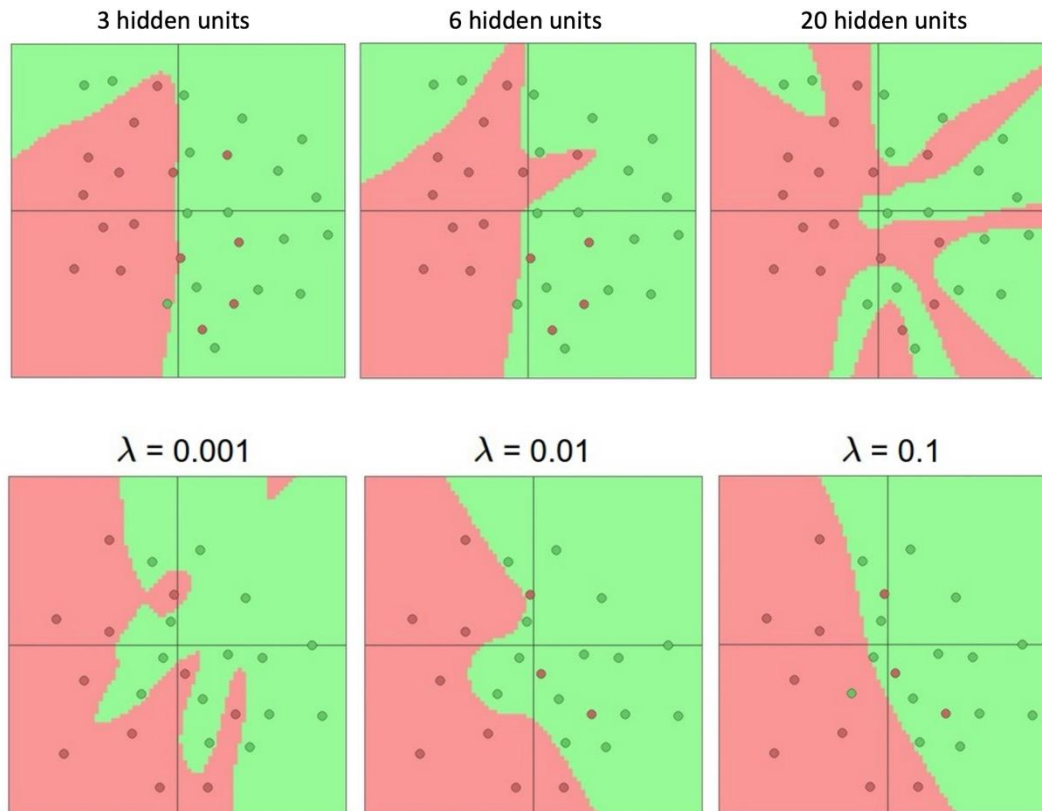
Not linearly separable in feature space



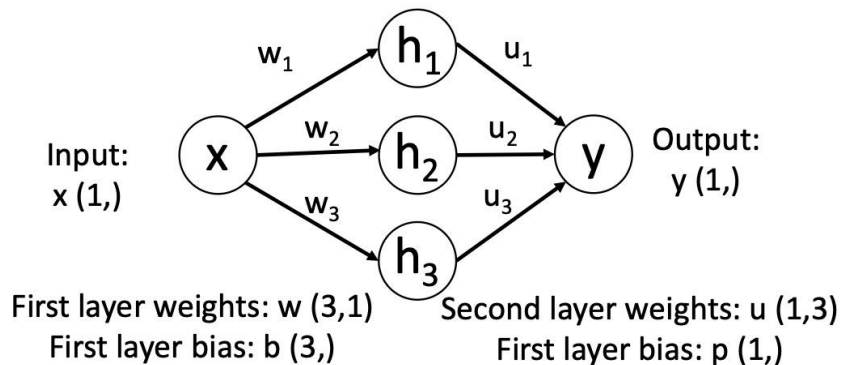
Space Warping



Space Warping



Universal Approximation



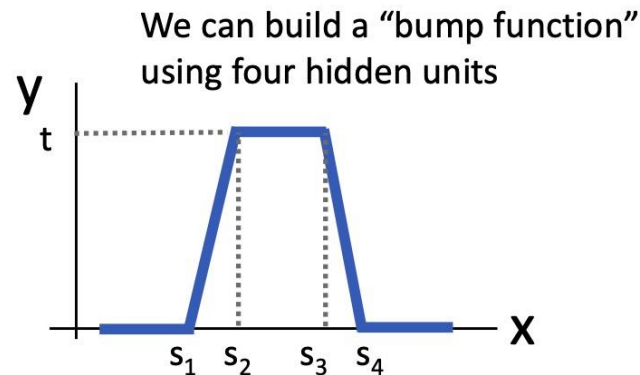
$$h_1 = \max(0, w_1 * x + b_1)$$

$$h_2 = \max(0, w_2 * x + b_2)$$

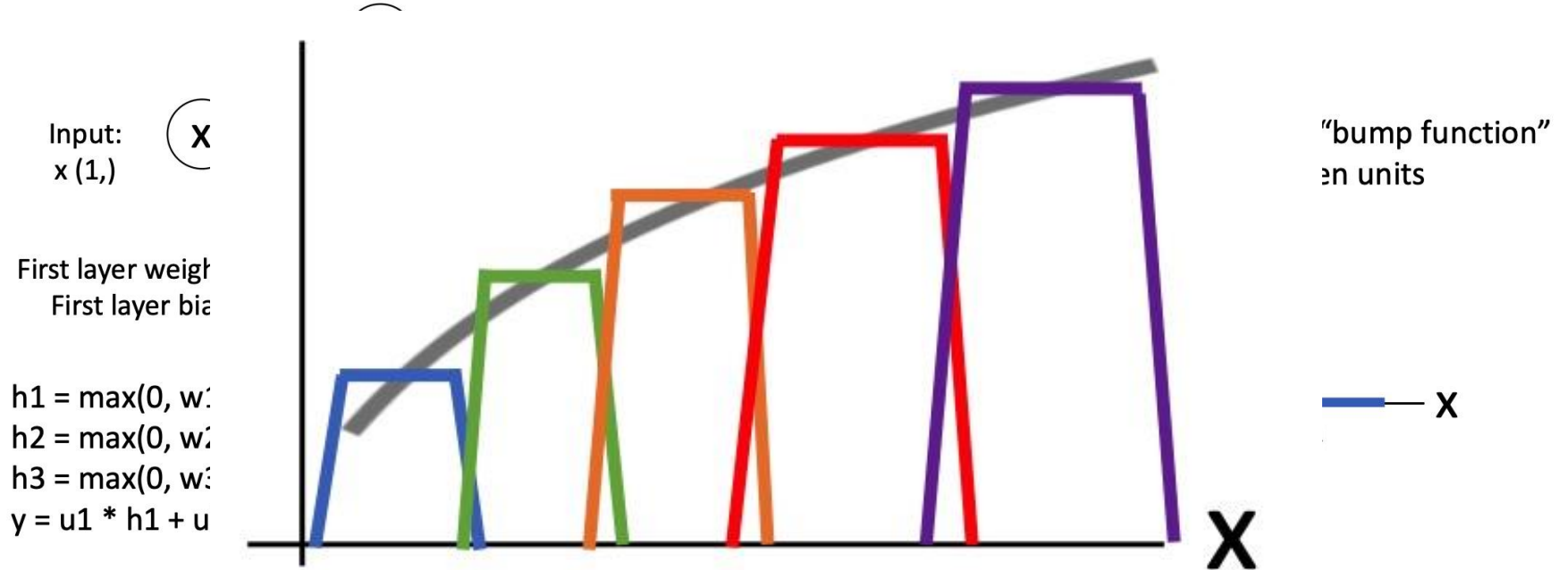
$$h_3 = \max(0, w_3 * x + b_3)$$

$$y = u_1 * h_1 + u_2 * h_2 + u_3 * h_3 + p$$

$$\begin{aligned} y = & u_1 * \max(0, w_1 * x + b_1) \\ & + u_2 * \max(0, w_2 * x + b_2) \\ & + u_3 * \max(0, w_3 * x + b_3) \\ & + p \end{aligned}$$



Universal Approximation

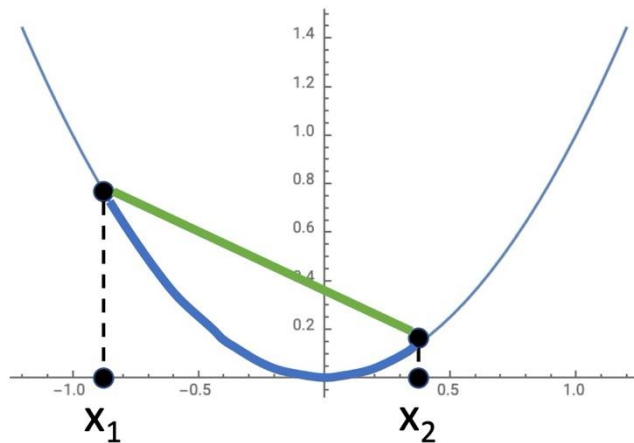


Convex Functions

A function $f : X \subseteq \mathbb{R}^N \rightarrow \mathbb{R}$ is **convex** if for all $x_1, x_2 \in X, t \in [0, 1]$,

$$f(tx_1 + (1 - t)x_2) \leq tf(x_1) + (1 - t)f(x_2)$$

Example: $f(x) = x^2$ is convex:

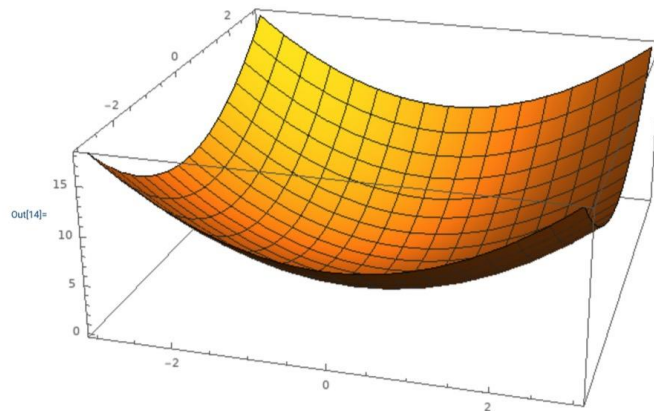


Convex Functions

A function $f : X \subseteq \mathbb{R}^N \rightarrow \mathbb{R}$ is **convex** if for all $x_1, x_2 \in X, t \in [0, 1]$,

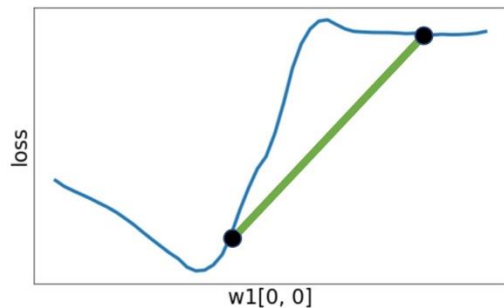
$$f(tx_1 + (1 - t)x_2) \leq tf(x_1) + (1 - t)f(x_2)$$

Intuition: A convex function
is a (multidimensional) bowl



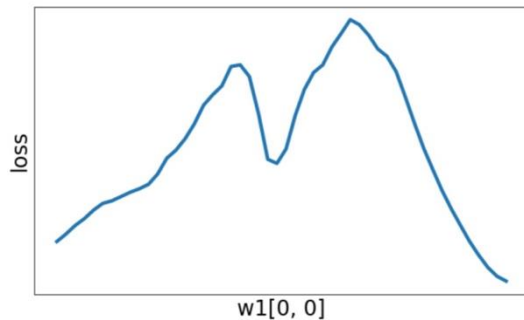
Convex Functions

But often clearly nonconvex:



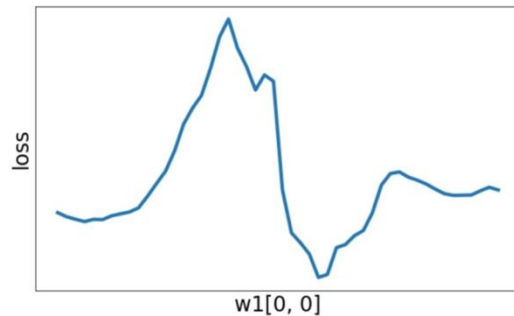
1D slice of loss landscape for a 4-layer ReLU network with 10 input features, 32 units per hidden layer, 10 categories, with softmax loss

With local minima:



1D slice of loss landscape for a 4-layer ReLU network with 10 input features, 32 units per hidden layer, 10 categories, with softmax loss

Can get very wild!

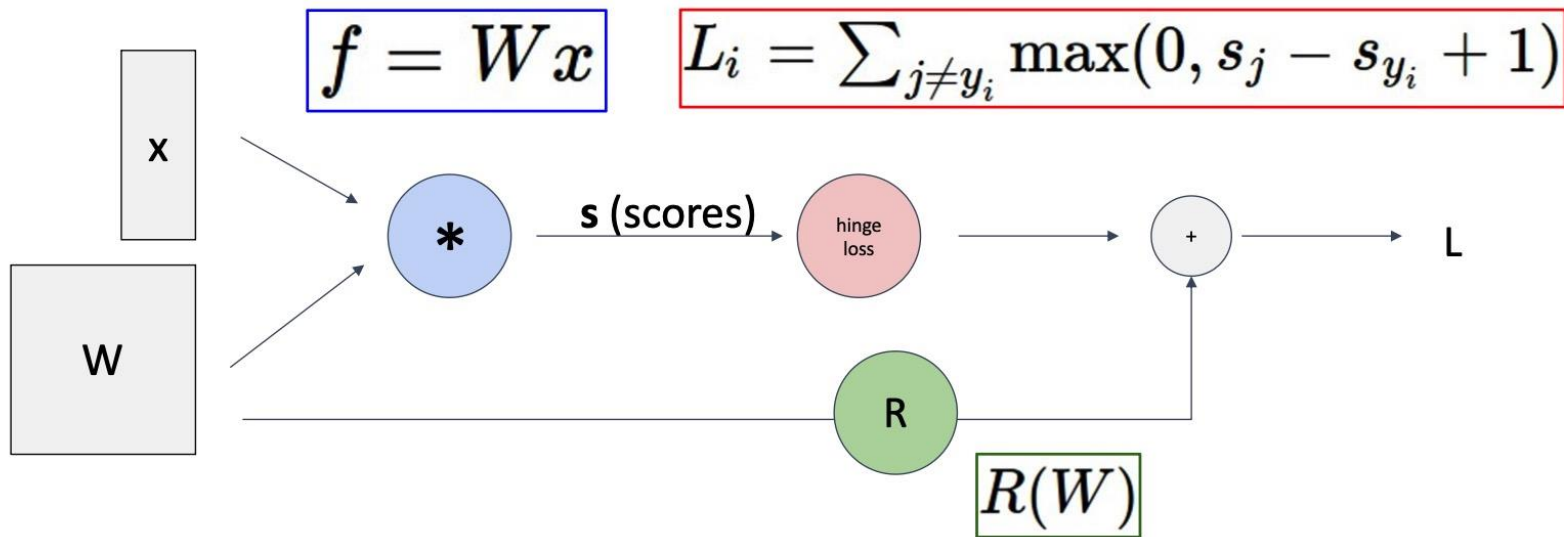


1D slice of loss landscape for a 4-layer ReLU network with 10 input features, 32 units per hidden layer, 10 categories, with softmax loss

Backpropagation

```
lookup.KeyValue  
f.constant(['em  
=tf.constant([G  
lookup.StaticV  
_buckets=5)
```

Compute Gradients



Backpropagation Example

$$f(x, y, z) = (x + y)z$$

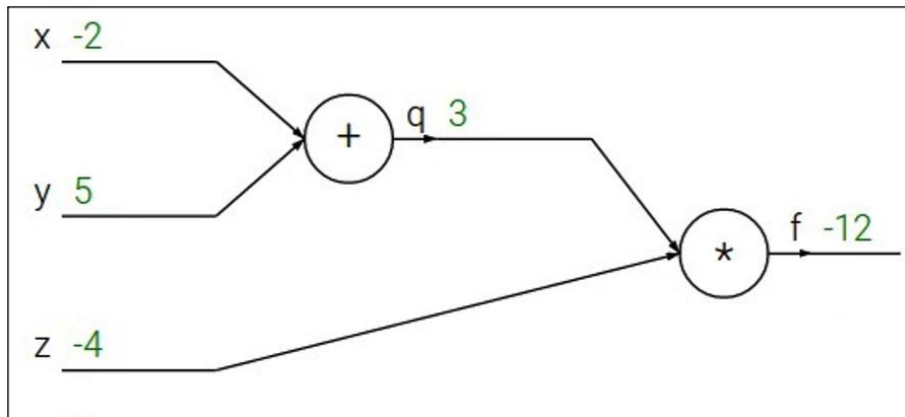
e.g. $x = -2, y = 5, z = -4$

1. Forward pass: Compute outputs

$$q = x + y \quad f = qz$$

2. Backward pass: Compute derivatives

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation Example

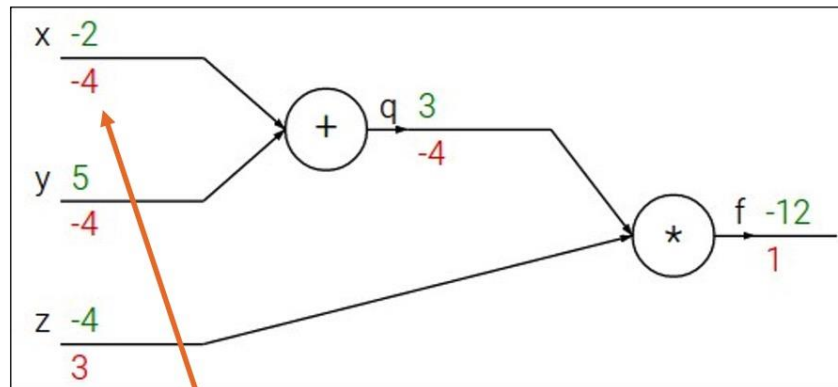
$$f = (x + y) \cdot z \Rightarrow q = x + y, \quad f = q \cdot z$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial x}$$

$$\bullet \frac{\partial f}{\partial q} = z$$

$$\bullet \frac{\partial q}{\partial x} = 1$$

$$\frac{\partial f}{\partial x} = z \cdot 1 = z$$



Chain Rule

$$\frac{\partial f}{\partial x} = \frac{\partial q}{\partial x} \frac{\partial f}{\partial q}$$

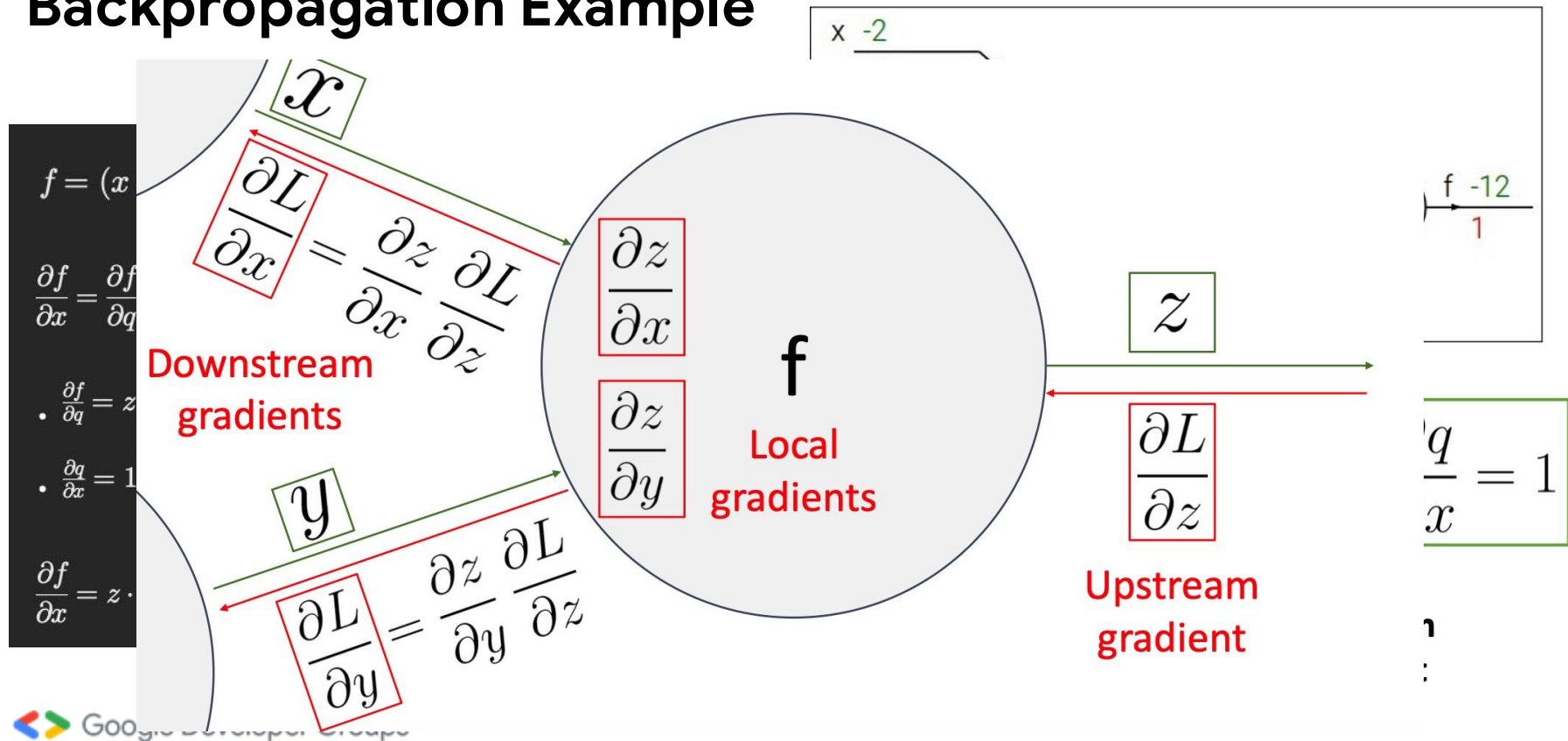
$$\frac{\partial q}{\partial x} = 1$$

**Downstream
Gradient**

**Local
Gradient**

**Upstream
Gradient**

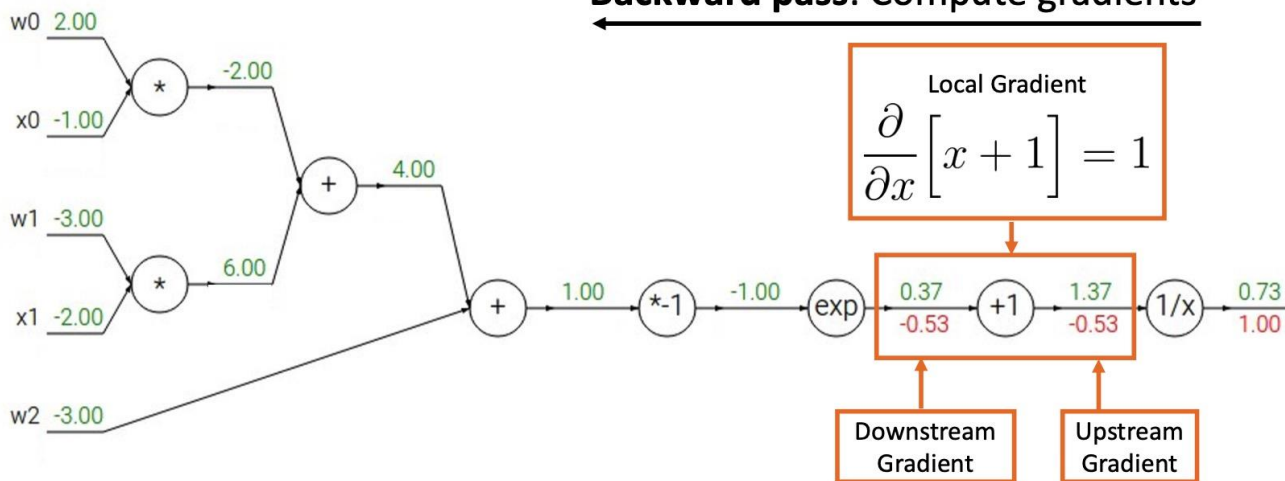
Backpropagation Example



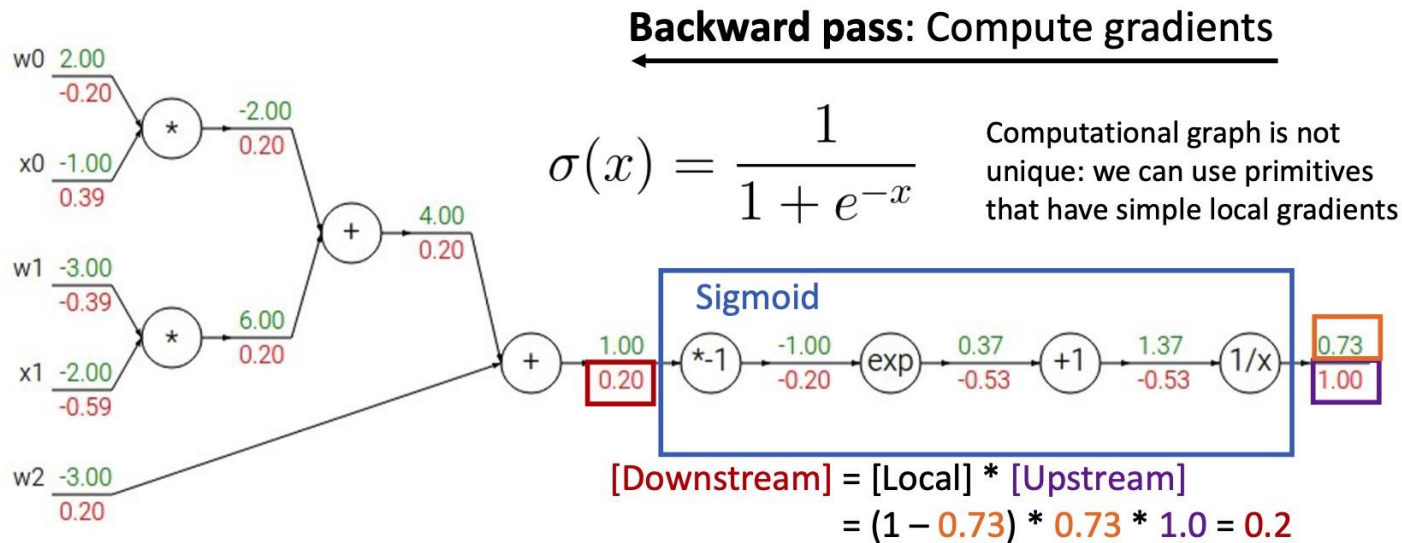
Backpropagation Example

Another Example $f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$

Backward pass: Compute gradients

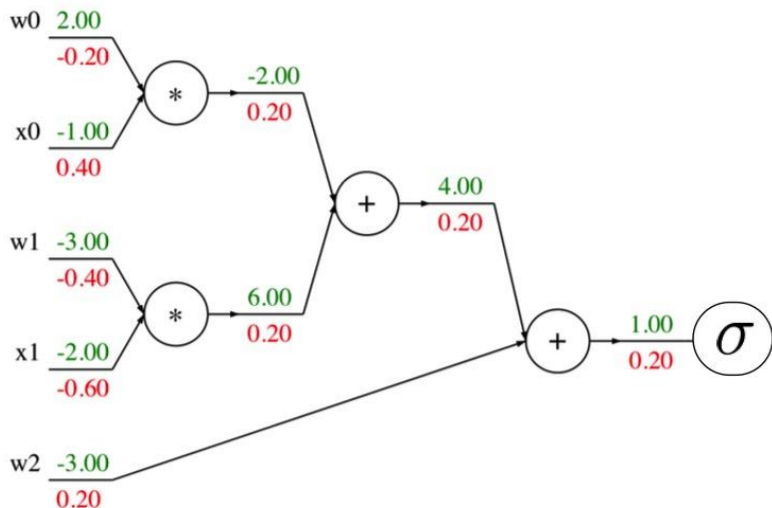


Backpropagation Example



Sigmoid local gradient: $\frac{\partial}{\partial x} [\sigma(x)] = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x))\sigma(x)$

Backpropagation Example



```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s2 = s0 + s1
```

```
    s3 = s2 + w2
```

```
    L = sigmoid(s3)
```

```
    grad_L = 1.0
```

```
    grad_s3 = grad_L * (1 - L) * L
```

```
    grad_w2 = grad_s3
```

```
    grad_s2 = grad_s3
```

```
    grad_s0 = grad_s2
```

```
    grad_s1 = grad_s2
```

```
    grad_w1 = grad_s1 * x1
```

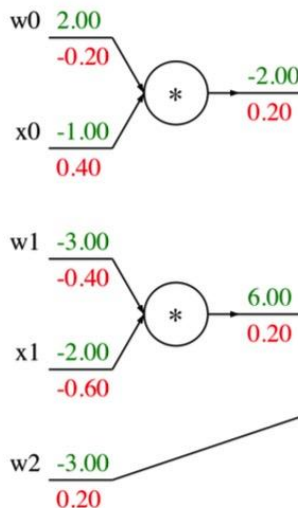
```
    grad_x1 = grad_s1 * w1
```

```
    grad_w0 = grad_s0 * x0
```

```
    grad_x0 = grad_s0 * w0
```

Sigmoid

Backpropagation Example



Gradient Flow 패턴 요약

조건	결과
입력이 작을 때 ($q \ll 0$)	$\sigma(q) \rightarrow 0$, gradient $\rightarrow 0$
입력이 클 때 ($q \gg 0$)	$\sigma(q) \rightarrow 1$, gradient $\rightarrow 0$
입력이 0 근처일 때	$\sigma(q) \approx 0.5$, gradient $\approx 0.25 \leftarrow$ 학습 잘 됨

```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s1
```

```
    w2
```

```
    d(s3)
```

```
    .0
```

```
    grad_L * (1 - L) * L
```

```
    grad_s3
```

```
    grad_s3
```

```
    grad_s2
```

```
    grad_s2
```

```
    grad_s1 * x1
```

```
    grad_x1 = grad_s1 * w1
```

```
    grad_w0 = grad_s0 * x0
```

```
    grad_x0 = grad_s0 * w0
```

Backpropagation with vector

$$x \in \mathbb{R}, y \in \mathbb{R}$$

Regular derivative:

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

If x changes by a small amount, how much will y change?

$$x \in \mathbb{R}^N, y \in \mathbb{R}$$

Derivative is **Gradient**:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^N \quad \left(\frac{\partial y}{\partial x} \right)_n = \frac{\partial y}{\partial x_n}$$

For each element of x , if it changes by a small amount then how much will y change?

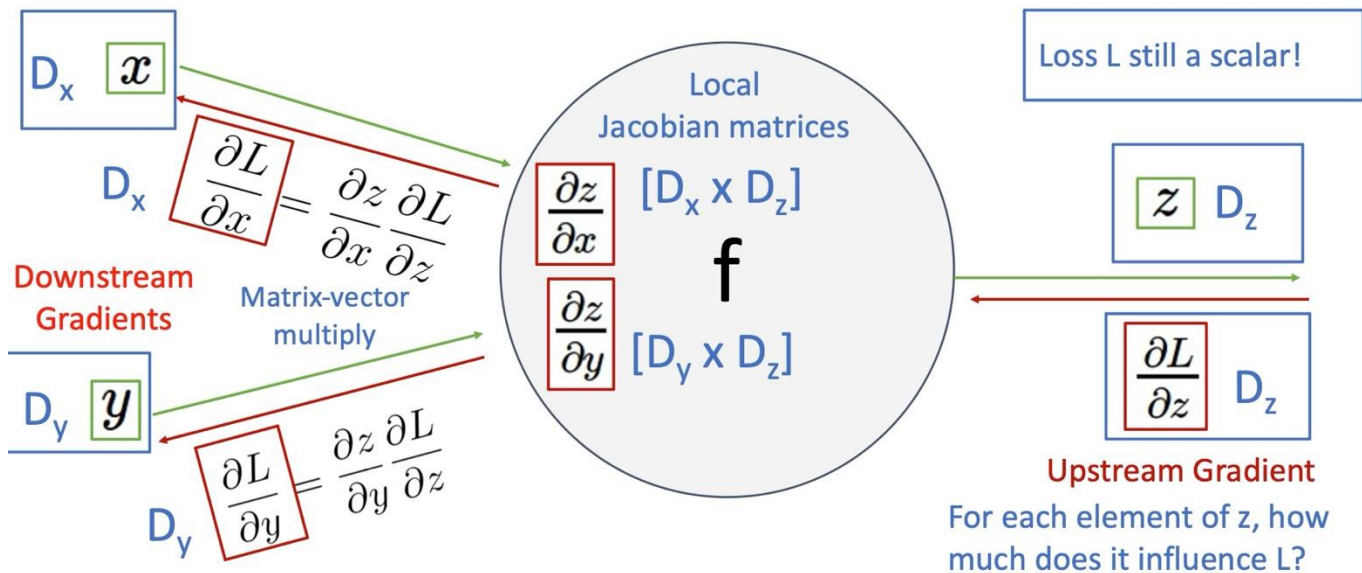
$$x \in \mathbb{R}^N, y \in \mathbb{R}^M$$

Derivative is **Jacobian**:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^{N \times M} \quad \left(\frac{\partial y}{\partial x} \right)_{n,m} = \frac{\partial y_m}{\partial x_n}$$

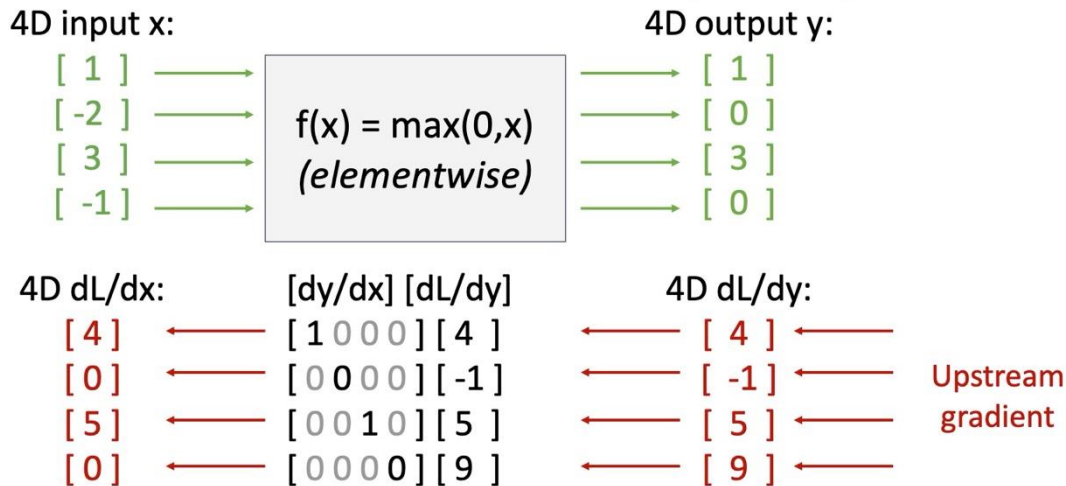
For each element of x , if it changes by a small amount then how much will each element of y change?

Backpropagation with vector

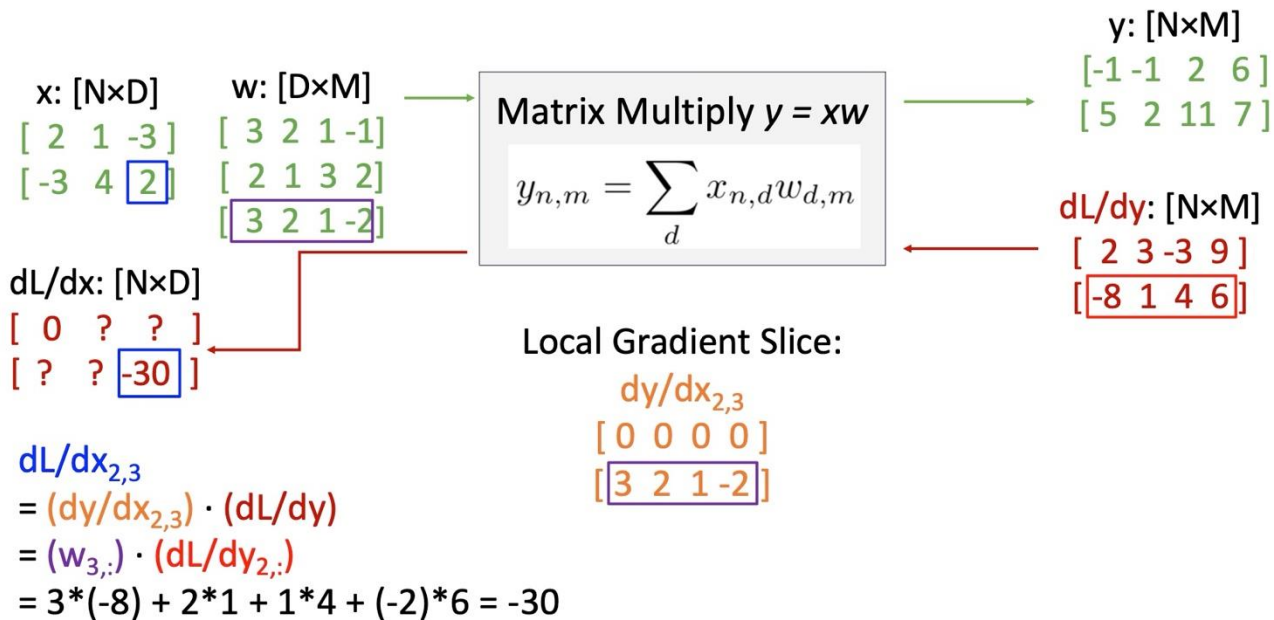


Backpropagation with vector

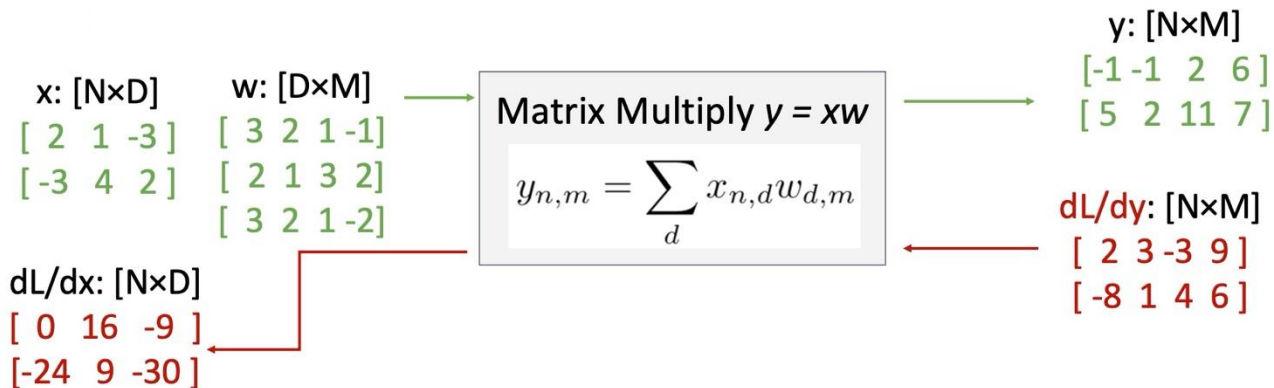
Jacobian is **sparse**: off-diagonal entries all zero! Never **explicitly** form Jacobian; instead use **implicit** multiplication



Backpropagation with matrix(tensor)

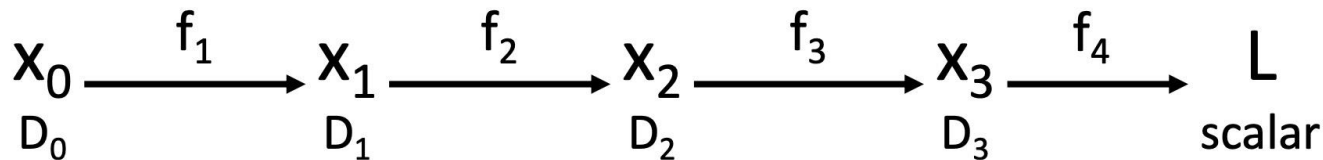


Backpropagation with matrix(tensor)



$$\begin{aligned} dL/dx_{i,j} &= (dy/dx_{i,j}) \cdot (dL/dy) \\ &= (w_{j,:}) \cdot (dL/dy_{i,:}) \end{aligned}$$

Backpropagation : Another View



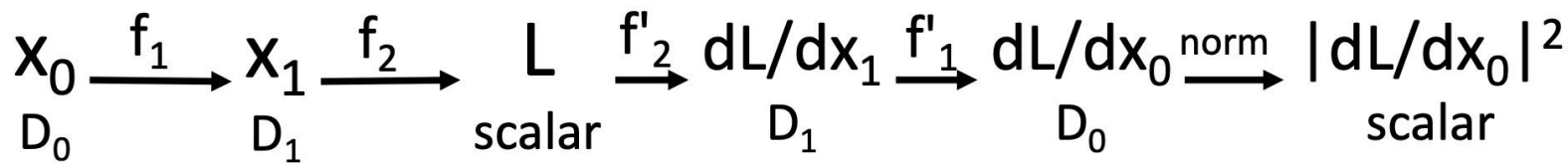
Matrix multiplication is **associative**: we can compute products in any order

$$\text{Chain rule } \frac{\partial L}{\partial x_0} = \underbrace{\left(\frac{\partial x_1}{\partial x_0} \right)}_{D_0 \times D_1} \underbrace{\left(\frac{\partial x_2}{\partial x_1} \right)}_{D_1 \times D_2} \underbrace{\left(\frac{\partial x_3}{\partial x_2} \right)}_{D_2 \times D_3} \underbrace{\left(\frac{\partial L}{\partial x_3} \right)}_{D_3}$$

Reverse-Mode Automatic Differentiation & Forward-Mode Automatic Differentiation

항목	Forward-mode	Reverse-mode
목적	여러 출력 각각의 기울기	하나의 출력에 대한 모든 입력의 기울기
계산량	출력 수만큼 반복	입력 수가 많을 때 효율적
딥러닝에 적합?	❌ (비효율)	✅ (Loss는 scalar, 파라미터는 수백만 개)

Backprop: Higher-Order Derivatives



Backprop!

This is implemented in PyTorch / Tensorflow!

Example: Regularization to penalize the norm of the gradient

$$R(W) = \left\| \frac{\partial L}{\partial W} \right\|_2^2 = \left(\frac{\partial L}{\partial W} \right) \cdot \left(\frac{\partial L}{\partial W} \right) \quad \frac{\partial}{\partial x_0} [R(W)] = 2 \left(\frac{\partial^2 L}{\partial x_0^2} \right) \left(\frac{\partial L}{\partial x_0} \right)$$